1 研究背景与目前系统现状

1.1 研究背景

对于蛋白质设计这类高代价任务而言，系统真正面对的难点不只是”是否会调用工具“，而在于以下几个方面：

- 任务链条长，且阶段之间存在强依赖关系；
- 不同阶段的计算代价和失败后果高度不对称；
- 运行过程中常常出现信息不充分、候选分歧、局部失败和恢复决策问题；
- 何时继续自动执行、何时切换策略、何时请求人工介入，本身就是关键问题。

因此，如果将毕设继续定义为“蛋白质设计助手”，其研究重点很容易停留在“任务分解、工具调用、结果输出”这一层面，即便加入了风险门控等系统。如果希望体现更明确的计算机相关的专业特征的话，我觉得可以把问题进一步上升为“高代价科学工作流的执行控制与治理”。

1.2 当前系统现状

从现有的系统设计来看，它已经形成了一个较为稳定的多 Agent 工作流执行系统。

首先，系统采用了显式有限状态机（FSM）作为运行核心，要求任务状态显式、状态转移合法且可记录，外部语义状态和内部执行状态均有明确定义。这意味着系统已经具备较好的执行一致性、可恢复性与可审计性基础。

其次，系统在角色划分上已经形成了较为清晰的职责边界：PlannerAgent 负责生成计划候选，ExecutorAgent 负责工具执行和恢复流程，SafetyAgent 负责评估，SummarizerAgent 负责结果汇总。这种结构表明系统已经具备典型多 Agent runtime 的雏形。

再次，系统已经围绕蛋白质设计任务构建了较为完整的阶段化流程，已经支持：

- S1 序列候选生成
- S2 结构投影
- S3 质量门控
- S4 基于结构条件的细化
- 分层 patch/replan 恢复机制

此外，系统已经具备 HITL（Human-in-the-loop）机制和恢复链设计，包括 `WAITING_*` 以及 $retry \to patch \to replan$ 的恢复顺序要求。这说明系统已经在工程层面充分考虑了可恢复执行和人工参与的问题。

最后，系统已经具备一定实验与评测基础。当前仓库中存在 A0-A6 纵向实验框架，用于从效果、机制、代价、治理维度评估系统表现，并支持复现与增量归因分析。这为后续研究提供了较好的实验基础。

1.3 对当前系统的整体评估

综合来看，当前系统已经具备：

- 较成熟的多 Agent runtime 骨架；
- 显式状态机与和恢复执行机制；
- 蛋白质设计的具体工作流场景；
- 实验框架和评测口径

但从毕设角度考虑，该系统也存在一个更关键的问题：其更像是一个完成度较高的执行平台，而不是一个核心算法问题已经被明确抽象出来的研究课题。

这一问题主要体现在三个方面。

第一，当前系统虽然已经具备打分、风险等级、代价评估等概念，但从实现上来看，这些更多是启发式聚合结果，主要服务于候选排序和展示，尚未形成一个足够独立、足够可论证的核心算法。

第二，系统当前的主要研究对象仍然是“生命科学任务 → 多 Agent 规划执行 → 工具调用 → 结果输出”。这意味着，即便加入了门控、人工决策、patch/replan 等机制，它在选题层面仍然会被看成“生命科学助手”的一部分或一部分的增强版，而非一个独立成立的计算机算法课题。

第三，当前系统的许多优点更多体现在“工程正确性”和“执行健壮性”上面。例如状态一致性、审计、恢复、人工确认等。这些无法突出算法上的创新，因此必须进一步回答：系统究竟依据什么做出关键治理决策，如何量化任务难度、执行风险和工作量，并据此改变执行路径。

小结 因此，对当前系统的总体判断可以概括为：当前系统已经具备成为实验平台和实验载体的条件，但是尚未形成一个足够清晰、足够独立的核心算法主线。它最大的风险是“研究对象还没有从生命科学助手（或蛋白质设计平台）提升为更底层、更通用的计算机问题”。如果不进行这一提升，那么即便继续为系统添加功能，也仍然可能被视为另一生命科学多 Agent 助手系统的补充或子集，而难以形成一个边界清晰、创新点明确的毕业设计选题。

2 当前选题面临的核心问题

在已有系统的基础上，当前毕设如果继续按原有思路推荐，其主要问题不在于“系统功能不够丰富”，而在于选题主线尚未完成从应用系统到研究问题的提升。这使得当前选题在创新性、独立性和答辩的可解释性方面都存在明显风险。

1. 缺少一个足够明确的核心算法问题 目前系统已经包含多 Agent 协作、显式状态机、HITL、`patch/replan`、蛋白质设计工作流等多个组成部分，但这些组成部分更多体现为一个较完整的执行平台，而不是一个被单独抽象和证明的算法对象。

换句话说，当前系统解决了“如何把流程跑起来”的问题，却没有清晰回答“系统依据什么原则做关键决策”这一更为核心的问题。尤其在以下方面，仍然缺少关键建模：

- 如何刻画任务难度
- 如何估计执行风险
- 如何衡量工作量与代价
- 如何在不同风险和预算条件下调整执行策略
- 如何决定是否值得自动执行、请求人工接入或直接恢复/终止

虽然当前已有部分门控和打分机制，但更多属于启发式聚合和排序，而不是一个足够独立、可被论证为“核心算法”的研究对象。

因此，如果按照当前路径继续推进，很可能被评价为缺乏足够集中、足够有辨识度的算法贡献。

2. 当前选题仍然容易被归入“生命科学助手”子集 已经有另一个同学的毕设是“生命科学助手”，同样采用多 Agent 架构。从架构相似性上来说，即便本系统已经具备一些机制，也很难天然摆脱“生命科学助手系统的一种更为复杂的实现”这一印象。

原因在于，当前系统的主要叙事仍然是：

用户提出生命科学任务 → 多 `Agent` 规划与执行调工具 → 返回结果

只要这个叙事不变，那么无论内部运行机制如何增强，它仍然有较大概率被看作生命科学助手的执行引擎、后端工作流或子模块。这对答辩会有很大风险。

因为该系统很难回答与已有的生命科学助手的本质区别、核心贡献是什么等这些问题。

3. 现有的增强机制更偏向工程健壮性，而非算法创新 当前系统中已经存在许多机制保证实验平台能力，但是从选题的角度去看，它们更多体现的是系统设计的正确性、运行治理等方面，没有进一步上升为统一的决策原理和算法问题。

4. 难度、工作量、风险尚未进入统一决策 此前反复讨论的关键词包括难度、工作量、风险、不确定性、成本和人工介入时机等，这些词都指向一个共同问题：系统缺少的不是更多功能，而是一个能够解释并驱动关键治理动作的统一决策框架。

如果没有这一框架，那么就无法体现为什么某个任务更难、为什么某个任务需要人工介入、为什么某条恢复链路比另一条更合适等问题。

小结 因此，当前选题面临的核心问题可以概括为三点：

第一，系统已经具备较完善的平台能力，但是尚未抽象出一个足够独立的核心算法问题。

第二，只要主叙事仍然停留在“生命科学领域的多 Agent 助手”，就难免被已经存在的父集系统覆盖。

第三，在当前时间约束下，必须尽快把研究对象从“继续扩展系统功能”转向“提炼一个可论证、可验证、可答辩”的核心问题。

也正因如此，后续的研究方向应该不再是“再加入什么能力”，而是究竟把当前系统上升为哪一个更为底层和独立的计算机专业课题的问题。

3 选题目标的重新定义

在前两部分的分析之后，可以看到，当前问题的关键不在于系统是否已经足够复杂，也不在于是否可以继续增加更多功能，而在于现有选题的研究对象需要重新定义。如果这一点不改变，那么后续的所有改进都很可能会继续停留在“把生命科学领域的多 Agent 助手做得更完善”这一层面上，而难以形成一个边界清晰、创新点集中、能独立成立的毕业设计选题。

因此，下一步重要的不是继续扩展原系统，而是转换到研究问题上。

从原有思路看，课题更容易被理解成：构建一个面向生命科学任务（蛋白质设计任务）的多 Agent 系统，通过规划、执行、工具调用和人工确认，帮助完成蛋白质设计任务等科研任务。这种表述的主语仍然是“生命科学助手”。一旦如此，无论其内部加入多少与运行机制，都仍然可能被视为一个更大助手系统中的一部分。也就是说，问题不在于这些机制本身是否重要，而在于它们没有被提升为一个更底层的研究对象。

要解决这一点，课题目标必须从“做一个生命科学助手”转向“研究高代价科学工作流如何被控制”。这一变化在明确毕设的边界。一旦采用新的表述，系统的核心问题就不再是“如何理解生命科学任务并给出结果”，而变成了“在一个代价高、链路长、存在失败与恢复、且允许人工介入的科学工作流中，系统应该依据什么原则决定下一步如何推进”。这样，系统关注的重点也会发生变化：难度、风险、恢复空间、人工介入时机等因素，不再是辅助性的描述目标，而是会成为驱动执行策略变化的核心决策变量。课题的重点就从一个助手或系统，转变成运行时控制与治理机制。

我认为这一重新定义是重要的，因为它同时解决了三个问题：

首先，它能够把课题从一个应用层面的生命科学助手，提升为一个更加通用的计算机问题。只要研究对象被定义成“高代价科学工作流控制”，那么蛋白质设计就不再是课题本体，而只是一个验证场景。这样做的意义在于，系统的核心方法理论上不仅可以服务于蛋白质设计，也可以扩展到其他科学工作流场景，例如复杂文献调研、材料设计或多阶段实验分析。也就是说，课题不再被一个具体应用锁死，而开始具备“方法先于场景”的结构。

其次，这种重新定义有助于切断与“生命科学助手”父集之间的包含关系。如果别人的系统关注的是“如何面向生命科学系统提供智能帮助”，而当前课题关注的是“如

何控制高代价工作流的执行、恢复与人工介入”，那么两者的层级就已经不同。前者关注任务入口和任务处理，后者关注治理逻辑。此时，生命科学助手可以被视为该课题的上层入口，而不是父集。这种关系转换对于选题的独立性尤其重要。

再次，从计算机专业毕设的角度去看，这种重新定义也容易形成明确的算法主线。一旦将命题定义为“工作流控制”，就必须回答一系列典型的计算机问题：系统状态如何表示，哪些变量是不可直接观测的，运行时观测如何更新系统判断，动作空间如何定义，目标函数如何刻画成本、风险和收益之间的平衡，以及在有限预算和不确定性条件下系统如何做出决策。这些问题天然带有控制、调度、约束优化和序贯决策的特征，明显比“继续扩展生命科学助手能力”更接近一个可论证的研究方向。

在这一基础上，新的目标可以概括为：**不再把毕设定义为生命科学多 Agent 助手，而是定义为面向高代价工作流的运行时治理与控制问题，并以蛋白质设计场景为验证对象**。这一目标意味着，现有系统的角色也在随之变化。它不再是毕设本身的全部内容，而是一个实验平台和运行载体。那么，该系统中采用的 FSM、HITL、patch/replan、候选与实验框架等，都不再是零散功能，而是成为后续控制算法可依附的基础环境。换句话说，真正的研究主题转移到该系统之上的运行时控制层。

另外比较重要的是，剩余的周期已经不足两个月。此时及时把目标收缩到“高代价科学工作流如何控制”这一更为聚焦的问题上，那么后续工作将更容易围绕一个明确的主线展开：系统状态如何抽象，难度和风险如何进入决策闭环，控制器如何决定继续执行、恢复、人工介入或终止，以及这种控制相对静态流程究竟能带来什么收益。这样一来，剩余时间更有可能形成一个完整且可答辩的结果。

小结 因此，当前毕设需要一次本质上的选题重构，即从“生命科学多 Agent 系统”转向“面向高代价科学工作流的运行时控制问题”。蛋白质设计仍是其重要的应用场景，但它不再是课题定义本身。只有完成这一层转换，后续关于创新点与可行性的讨论才具有意义。也只有在这一前提下，该课题才可能摆脱被父集系统覆盖的风险，成为一个边界清晰、问题集中、具备算法含量的毕设。

4 候选方向比较

在完成选题的重新定义之后，接下来的问题是在可选的研究方向中，哪一条最适合在当前时间、已有系统基础和答辩要求下收敛成一个成立的毕设课题。从目前的讨论来看，真正有代表性的方向大致可以分成四类：继续沿蛋白质设计系统增强路线推进、转向自动 benchmark、转向工作流编译/验证/修复，以及转向运行时 controller。它们分别对应不同的研究对象、不同的创新重心，也对应不同的风险。

4.1 延续“蛋白质设计系统增强”路线

这是与当前系统最接近的一条路线。其基本思路是继续围绕生命科学任务展开，将系统定义为一个面向蛋白质设计或更广义生命科学任务的多 Agent 助手，并再次基础

上不断补充能力，例如更完整的工作链、更复杂的任务拆解、更细致的门控、更丰富的 HITL 交互，以及更完善的恢复逻辑。

这一条路线的核心是：把系统当作应用产品或综合平台来完善。其关注重点在于系统覆盖了多少能力、能处理多少真实请求、交互链路是否顺畅、执行流程是否稳定。它的优势十分明显：与现有系统最贴近，实现成本最低，也最容易展示出成果。

但它的问题同样明显。首先，这条线路最容易被看成是另一个生命科学助手的自然扩展。因为只要系统的对外叙事仍然是“面向生命科学任务（蛋白质设计）的多 Agent 帮助系统”，那么内部无论加入多少状态机、恢复机制和人工确认逻辑，本质上都没有脱离“助手系统”这一概念。其次，这一条路径的创新点很难集中，往往会分散在多个工程增强点上，例如增加某一类工具、改进某一段流程或补充某种门控逻辑。这些内容单独来看都合理，但是很难压缩成一个明确的算法主线。最后，在当前时间压力下，继续走“大系统增强”的路线很容易导致内容越做越多，主线却越来越模糊。

因此，虽然这条路线工程上最平滑，但是从毕设选题的角度来看，它恰恰是风险最高的一条，因为它最容易落入无法解释核心研究问题是什么的局面。

4.2 自动 Benchmark 路线

自动 benchmark 路线的思路是把研究对象从“如何更好地完成任务”转向“如何系统地评测复杂科学工作流任务”。在这一方向上，系统不再首先关注“输出什么结果”，而是关注：如何构造具有不同复杂度和难度层级的任务，如何让这些任务可复现、可比较、可自动评分，以及如何让 benchmark 真正反映多 Agent 系统在高代价科学工作任务中的能力差异。

从近两年的研究趋势来看，这条路线是成立的，而且学术价值很明确。PaperBench(Starace et al., 2025) 强调的是研究复现实验的细粒度自动评分与 rubric 拆解，BrowseComp(Wei et al., 2025) 强调的是高难度信息寻求任务中的持久搜索，CoLLAB(Mahmud et al., 2025) 则突出协作复杂度和多 Agent benchmark 的可扩展构造。

这一线路的本质是：把“难度”本身变成研究对象。它不想助手系统那样关注“给出什么答案”，而是关注“什么样的任务更难、为什么难、如何被自动构造出来、又如何被可靠评分。”从独立性上来说，这条线路非常安全，因为 benchmark 不是生命科学助手的子模块，反而可以把不同助手都作为 benchmark 的被评测对象。从答辩角度看，也更容易回答“贡献到底是什么”，因为 benchmark 一旦设计得好，本身就是独立研究资产。

不过，这条线路本身也有自己的问题。首先，它的重点不是控制算法，而是在任务生成、难度分层和自动评分机制上。如果自动生成和自动评分不够强，benchmark 很容易退化成“整理了一批测试集”。其次，这条线路虽然能很好地体现“难度”，但是也相对弱化了与当前系统直接结合的优势。也就是说，它更独立，但是与现有系统的耦合不够。最后，从剩余时间来看，如果要同时做好任务构造、难度校准和轨迹级评分，工作量其实不小，切容易在“设计评测体系”这一块耗费过多时间。

因此，自动 benchmark 是一条边界很清晰、不容易成为子集的路线，但是它更像

是“独立评测平台”选题，和当前系统的天然结合度略低。

4.3 工作流编译、验证与修复路线

另一条可能的方向，是把研究结果定义为“从科研目标到可执行工作流的编译、验证与修复”。在这条路线下，重点不是任务本身，也不是最终结果，而是：如何从自然语言目标生成结构化工作流；如何检查工具链的 I/O 兼容性、参数合法性和资源约束；以及如何在执行失败后进行最小修复和后缀重规划。

从方法论上来看，这条路线非常有计算机专业特色，因为它天然带有 `compiler`、`verification`、`schema caching`、`plan repair` 等色彩。现有多 `Agent` 系统设计中，越来越多工作开始强调结构化任务表示、明确的状态机组织以及对工具链和交互协议的形式约束。例如 MetaAgent(Y. Zhang et al., 2025) 这类工作已经开始从显式状态机组织的角度构造多 Agent 系统，而工作流系统本身也越来越强调 plan schema、tool contract 和可验证执行过程。

这条线路的核心是：把科学任务的执行链看作一种可编译、可验证、可修复的程序结构。如果从理论表达上看，它很有吸引力。

但是它在当前语境下有两个显示问题。第一，与现有系统已有能力重合较多。当前系统已经具有显式 FSM、计划候选、patch/replan、候选验证和恢复机制，因此如果再把“工作流编译与修复”作为核心方向，很容易被看成对现有 `planner` 的进一步完善，而不是一个研究主线。第二，这条路线虽然很具有形式化，但是在叙事上较弱。换句话说，它更像“让流程更正确”，而不是“让系统在高代价下做出更好的运行时决策”。从应用意义和系统价值上，可能不够直观。

因此，工作流编译/验证/修复路线是一个理论性较强的候选方向，但是与当前系统的差异化空间并没有想象中大，而且更容易被视为现有 `planner` 和恢复逻辑的深化版本。

4.4 运行时 Controller 路线

`controller` 路线的出发点是把研究对象上升为：在高代价、长链路、带恢复与人工介入的科学工作流中，系统究竟如何决定下一步如何推进。在这一方向上，系统不再被理解成一个“完成生命科学任务的助手”，而被理解成一个“治理工作执行的运行时控制器”。

从近两年的研究来看，这条方向虽然还没有完全同构的现成答案，但是已经有多条重要脉络构成了它的理论背景。在不稳定性建模方面，不确定性思维（Uncertainty of Thoughts，UoT）说明不确定性可以进入 `Agent` 的规划闭环 (Hu et al., 2024)；在推理资源方面，BEST-Route 方法提出依据任务难度与质量阈值，对推理计算资源进行动态调度，实现性能与成本之间的权衡 (Ding et al., 2025)；在推理过程优化方面，QLASS 方法通过对中间决策状态进行价值估计，引导 Agent 在推理空间中的搜索路径，提高推理效率与结果质量 (Lin et al., 2025)；在行动与工具使用层面，EcoAct 从经济性约束出发，对 Agent 的动作选择与工具调用策略进行建模，强调资源受限环境下的最优决策问

题 (S. Zhang et al., 2024)；在多 Agent 系统层面，Why Do Multi-Agent LLM Systems Fail? 与 Which Agent Causes Task Failures and When? 的研究表明，系统失败并不完全来源于单一模型能力的不足，而与协作机制、验证流程、责任归因以及系统治理结构密切相关 (Cemri et al., 2025; S. Zhang et al., 2025)。

这一路线本质是：把“难度、风险、工作量和人工介入价值”统一纳入到一个运行时的决策问题。换句话说，`controller` 关注的不是任务内容本身，而是系统治理动作本身。它所决定的不是“下一句怎么回答”，而是诸如：

- 现在是否值得继续执行；
- 是否应该扩大候选探索
- 是否应切换到保守路径；
- 当前失败更适合 `patch` 还是 `replan`；
- 是否应请求 `HITL`
- 是否应在进一步浪费成本前提前终止

这使得 `controller` 方向同时具有三方面优势。第一，它与当前系统切合度最高。因为现有系统已经有状态机、HITL、patch/replan、阶段化执行链和实验框架，`controller` 可以自然地嵌入这些现有能力之上，而不需要推翻架构。第二，它最容易把“难度”从口头概念变成真正的算法变量。难度不再指示任务描述中的印象，而是需要在执行中动态估计、并据此改变策略的核心状态。第三，它在应用意义上最容易成立。蛋白质设计这类高代价场景天然就会遇到“什么时候继续算、什么时候该求助人类、什么时候必须止损”的问题，因此 `controller` 的价值不仅是理论上的，也非常符合真实科研流程的痛点。

当然，`controller` 方向也不是没有风险。它最大的风险在于，如果最后只做成“几个特征 + 一堆人工阈值 + 超阈值就进入 HITL”的规则系统，那么它虽然还能运行，但创新性会明显不足。因此这条路线真正成立的前提是：`controlle` 必须被明确建模为一个部分可观测、带风险与预算约束的序贯决策问题，而不是一个更复杂的门控模块。

4.5 不同方向的比较与理由

综合来看，这几个方向的差异可以概括为：生命科学助手最贴近现有系统，但独立性最弱，也最容易被已有父集覆盖；自动 benchmark 路线独立性最强，难度问题也最自然，但与现有系统直接结合较弱，且任务生成与自动评分部分在剩余时间内工作量也不小；工作流编译/验证/修复路线理论上较“计算机化”，但与现有 `planner` 的功能边界较近，容易被视为现有平台能力的深化；而 `controller` 路线则位于一个相对平衡的位置，它既能明显提升选题层次，使课题从“助手系统”转向“工作流治理”，又能最大限度复用现有系统作为实验平台，同时还能把“难度、风险、工作量、恢复和 HITL”这些此前分散的概念统一到一个核心决策问题中。

因此，如果在当前时间节点下选择一条既能体现算法性，又能形成明确主线、同时还能落到现有系统的路径，那么 controller 是最值得优先考虑的方向。而在 controller 内部，我准备进一步倾向于信念状态（Belief State)controller，因为它最适合把“难度”定义为一个运行时的隐变量，而不是一个静态标签或单纯的打分结果。

5 选择 Belief-State Controller

6 Belief-State Controller 问题定义

7 相关研究现状

8 可能的创新点、难点

9 可行性分析

10 预期结果

11 风险与备选方案

References

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent llm systems fail? https://arxiv.org/abs/2503.13657

Ding, D., Mallick, A., Zhang, S., Wang, C., Madrigal, D., Garcia, M. D. C. H., Xia, M., Lakshmanan, L. V. S., Wu, Q., & Rühle, V. (2025). BEST-route: Adaptive LLM routing with test-time optimal compute. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=tFBIbCVXkG

Hu, Z., Liu, C., Feng, X., Zhao, Y., Ng, S.-K., Luu, A. T., He, J., Koh, P. W., & Hooi, B. (2024). Uncertainty of thoughts: Uncertainty-aware planning enhances information seeking in large language models. https://arxiv.org/abs/2402.03271

Lin, Z., Tang, Y., Yao, X., Yin, D., Hu, Z., Sun, Y., & Chang, K.-W. (2025). QLASS: Boosting language agent inference via q-guided stepwise search. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=f6lio2CZIM

Mahmud, S., Bagdasarian, E., & Zilberstein, S. (2025). CoLLAB: A framework for designing scalable benchmarks for agentic LLMs. *Workshop on Scaling Environments for Agents*. https://openreview.net/forum?id=372FjQy1cF

Starace, G., Jaffe, O., Sherburn, D., Aung, J., Chan, J. S., Maksin, L., Dias, R., Mays, E., Kinsella, B., Thompson, W., Heidecke, J., Glaese, A., & Patwardhan, T. (2025). Paperbench: Evaluating ai's ability to replicate ai research. https://arxiv.org/abs/2504.01848

Wei, J., Sun, Z., Papay, S., McKinney, S., Han, J., Fulford, I., Chung, H. W., Passos, A. T., Fedus, W., & Glaese, A. (2025). *Browsecomp: A simple yet challenging benchmark for browsing agents*. https://arxiv.org/abs/2504.12516

Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., & Wu, Q. (2025). Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=GazlTYxZss

Zhang, S., Zhang, J., Ding, D., Garcia, M. H., Mallick, A., Madrigal, D., Xia, M., Rühle, V., Wu, Q., & Wang, C. (2024). Ecoact: Economic agent determines when to register what action. https://arxiv.org/abs/2411.01643

Zhang, Y., Liu, X., & Xiao, C. (2025). Metaagent: Automatically constructing multi-agent systems based on finite state machines. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=vOxaD3hhPt